

Lab4db

May 14, 2026

1 Tableau and Schur functions

1.1 Math 737 - Lab 4

1.2 SYT and SSYT are classes of objects in Sage. Evaluate the cells below to see how to do some things with SYT. Then in Exercise 2, you'll do similar things with SSYT.

```
[7]: S = StandardTableaux([3,3])
```

```
for tab in S:  
    display(tab)
```

1	3	5
2	4	6

1	2	5
3	4	6

1	3	4
2	5	6

1	2	4
3	5	6

1	2	3
4	5	6

```
[9]: StandardTableaux([6,5,4,3,2,1]).cardinality()
```

[9]: 1100742656

1.3 Exercise 1a: How many SYT are there of shape $[n, n]$, for n ranging from 1 to 8? What numbers are these?

1.3.1 Hint: Use a ‘for’ loop. An example of a ‘for’ loop is given below.

```
[12]: YT_nn=[]
      for i in range(1,9):
          print(f"number of 2-by-{i} Young Diagrams: {StandardTableaux([i,i]).
            ↪cardinality()}")
          YT_nn.append(StandardTableaux([i,i]).cardinality())
      print(f"Sequence: {YT_nn}")
```

```
number of 2-by-1 Young Diagrams: 1
number of 2-by-2 Young Diagrams: 2
number of 2-by-3 Young Diagrams: 5
number of 2-by-4 Young Diagrams: 14
number of 2-by-5 Young Diagrams: 42
number of 2-by-6 Young Diagrams: 132
number of 2-by-7 Young Diagrams: 429
number of 2-by-8 Young Diagrams: 1430
Sequence: [1, 2, 5, 14, 42, 132, 429, 1430]
```

These are the the Catalan numbers. $C_n = \# \text{SYT}[n, n]$

```
[11]: for n in [1..8]:
      S1 = StandardTableaux([n,n])
      S1.cardinality()
      print("----")
```

```
----
----
----
----
----
----
----
----
```

1.4 Exercise 1b: Use the command `SemistandardTableaux` to create (not print!) the set of SSYT there are of shape `[3,3]` (with entries bounded above by 6). How many tableaux are in this set?

1.5 Then use the optional parameter `max_entry` to create the set of SSYT of shape `[3,3]` with entries bounded above by 3. Print this smaller set of SSYT.

1.5.1 (If you need help with this, evaluate the cell below to see the documentation.)

```
[15]: SSYT = SemistandardTableaux([3,3])
      SSYT.cardinality()
```

```
[15]: 490
```

```
[17]: SSYT = SemistandardTableaux([3,3], max_entry=3)
      SSYT.cardinality()
```

```
[17]: 10
```

1.6 Evaluate the cells below to see how to create a SSYT and row-insert numbers into it.

```
[22]: T = SemistandardTableau([[1,2,4,11],[3,5,9,12],[6,8],[7],[10]])
      print("T:")
      display(T)
```

T:

1	2	4	11
3	5	9	12
6	8		
7			
10			

```
[27]: print("T<-(4)")
      display(T.schensted_insert(4))
```

T<-(4)

1	2	4	4
3	5	9	11
6	8	12	
7			
10			

```
[20]: w = T.to_word()
      w
```

[20]: word: 10,7,6,8,3,5,9,12,1,2,4,11

```
[28]: U = SemistandardTableau([[1,2],[3]])
print("T<-")
display(T.bump_multiply(U))
```

T<-

1	1	2	11
2	3	9	12
3	4		
5	8		
6			
7			
10			

1.7 Exercise 2: Use the above commands to verify the examples of SSYT multiplication you did in class in two ways: Write a ‘for’ loop to Schensted insert the letters of the word of the tableau one at a time to create a SSYT. Then, show that you get the same thing when you use `bump_multiply`.

```
[71]: T = SemistandardTableau([[1,2,4,11],[3,5,9,12],[6,8],[7],[10]])
U = SemistandardTableau([[1,2],[3]])
print("T:")
display(T)
print("U:")
display(U)
```

T:

1	2	4	11
3	5	9	12
6	8		
7			
10			

U:

1	2
3	

Loop Result

```
[73]: u = U.to_word()
print(u)
TU=T
```

```

for l in u:
    TU = TU.schensted_insert(l)

display(TU)

```

312

1	1	2	11
2	3	9	12
3	4		
5	8		
6			
7			
10			

Multiplication result agrees:

```
[68]: display(T.bump_multiply(U))
```

1	1	2	11
2	3	9	12
3	4		
5	8		
6			
7			
10			

- 1.8 Exercise 3a: Use the command `SemistandardSkewTableaux` to create (not print!) the set of skew SSYT shape λ/μ with $\lambda = [4, 4, 2, 1]$ and $\mu = [2, 1]$ with entries bounded above by 4. How many tableaux are in this set?
- 1.9 Now create the set of skew SSYT of shape λ/μ with $\lambda = [3, 3]$ and $\mu = [2, 1]$ with entries bounded above by 3. Use a ‘for’ loop to print this smaller set of SSYT.
- 1.9.1 (Recall, if you need help, you can type a command followed by `?` to see the documentation.)

```

[104]: SSskew1 = SemistandardSkewTableaux([[4,4,2,1],[2,1]], max_entry=4)
print(f"# of skew tableau with shape [4,4,2,1]/[2,1]: {SSskew1.cardinality()}")

SSskew2 = SemistandardSkewTableaux([[3,3],[2,1]], max_entry=3)
for tab in SSskew2:
    tab.pp()

```

```

# of skew tableau with shape [4,4,2,1]/[2,1]: 664
. . 1
. 1 2
. . 1

```

```

. 1 3
. . 1
. 2 2
. . 2
. 1 3
. . 1
. 2 3
. . 1
. 3 3
. . 2
. 2 3
. . 2
. 3 3

```

1.10 Evaluate the cells below to see how to create a specific skew SSYT and rectify it.

```

[111]: T = SkewTableau([[None, None, 2, 7], [None, 3, 5, 8], [6, 8], [7], [8]])
print("Skew Tableau")
T.pp()
print("Rectified:")
(T.rectify()).pp()

```

Skew Tableau

```

. . 2 7
. 3 5 8
6 8
7
8

```

Rectified:

```

2 5 7
3 8 8
6
7
8

```

1.11 Exercise 3b: In Exercise 2, you saw how to multiply SSYT. Do one of these same multiplications another way: Use the above commands to create the appropriate skew SSYT so that rectification of that skew SSYT yields the multiplication of the two tableaux.

```

[116]: T = SemistandardTableau([[1, 2, 4, 11], [3, 5, 9, 12], [6, 8], [7], [10]])
U = SemistandardTableau([[1, 2], [3]])

# build skew tableau by placing U northeast of T
skewTU = SkewTableau([
    [None, None, None, None, 1, 2],

```

```

[None, None, None, None, 3],
[1, 2, 4, 11],
[3, 5, 9, 12],
[6, 8],
[7],
[10]
])
print("Skew tableau:")
skewTU.pp()

print("\nRectification:")
(skewTU.rectify()).pp()

print("\nComputed Product:")
TU.pp()

```

Skew tableau:

```

. . . . 1 2
. . . . 3
1 2 4 11
3 5 9 12
6 8
7
10

```

Rectification:

```

1 1 2 11
2 3 9 12
3 4
5 8
6
7
10

```

Computed Product:

```

1 1 2 11
2 3 9 12
3 4
5 8
6
7
10

```

1.12 Exercise 4: Is SSYT multiplication commutative? Prove or give a counterexample.

```
[119]: # Exercise 4
T = SemistandardTableau([[1,2,4,11],[3,5,9,12],[6,8],[7],[10]])
U = SemistandardTableau([[1,2],[3]])
print("T:")
T.pp()
print("U:")
U.pp()
print("TU:")
TU.pp()
print("UT:")
UT.pp()
print(f"We can see the two are not the same, but also to computationally verify_
↳the result of TU=UT is: {TU == UT}")
```

T:

```
1  2  4 11
3  5  9 12
6  8
7
10
```

U:

```
1  2
3
```

TU:

```
1  1  2 11
2  3  9 12
3  4
5  8
6
7
10
```

UT:

```
1  1  2  4  9 11
2  3  5 12
3  6  8
7
10
```

We can see the two are not the same, but also to computationally verify the result of TU=UT is: False

So we see multiplication is not commutative.

1.13 Exercise 5: Evaluate the blocks below to see how to work with the built-in Schur polynomial function in Sage. Verify that you got the same polynomial as we evaluated in class yesterday. Then try changing the parameters to compute other Schur polynomials.

```
[135]: s = SymmetricFunctions(QQ).s()
print(s([2,1]).expand(3))
print("Next: ")
print(s([5,2]).expand(2))
print("Next: ")
print(s([1,1]).expand(6))
```

$x_0^2x_1 + x_0x_1^2 + x_0^2x_2 + 2x_0x_1x_2 + x_1^2x_2 + x_0x_2^2 + x_1x_2^2$

Next:

$x_0^5x_1^2 + x_0^4x_1^3 + x_0^3x_1^4 + x_0^2x_1^5$

Next:

$x_0x_1 + x_0x_2 + x_1x_2 + x_0x_3 + x_1x_3 + x_2x_3 + x_0x_4 + x_1x_4 + x_2x_4 + x_3x_4 +$
 $x_0x_5 + x_1x_5 + x_2x_5 + x_3x_5 + x_4x_5$

1.14 Exercise 6 (if time): Play with the built-in tableaux functions to find some fun things to do with tableaux. Pick your favorite, demonstrate using it, and write a sentence to say what it does.

```
[128]: A = SemistandardTableau([[1,2,3,4],[5,6,7],[8,8],[35]])
B = SemistandardTableau([[1,2],[3,4]])
```

```
# build skew tableau by placing U northeast of T
skewAB = SkewTableau([
    [None, None, None, None, 1, 2],
    [None, None, None, None, 3, 4],
    [1, 2, 3, 4],
    [5, 6, 7],
    [8, 8],
    [35]
])
print("Skew tableau:")
skewAB.pp()

print("\nRectification:")
(skewAB.rectify()).pp()

print("\nProduct:")
(A.bump_multiply(B)).pp()
```

Skew tableau:

```

. . . . 1 2
. . . . 3 4
1 2 3 4
```

5 6 7
8 8
35

Rectification:

1 1 2 3 4
2 3 7
4 6
5 8
8
35

Product:

1 1 2 3 4
2 3 7
4 6
5 8
8
35

I like the rectification stuff